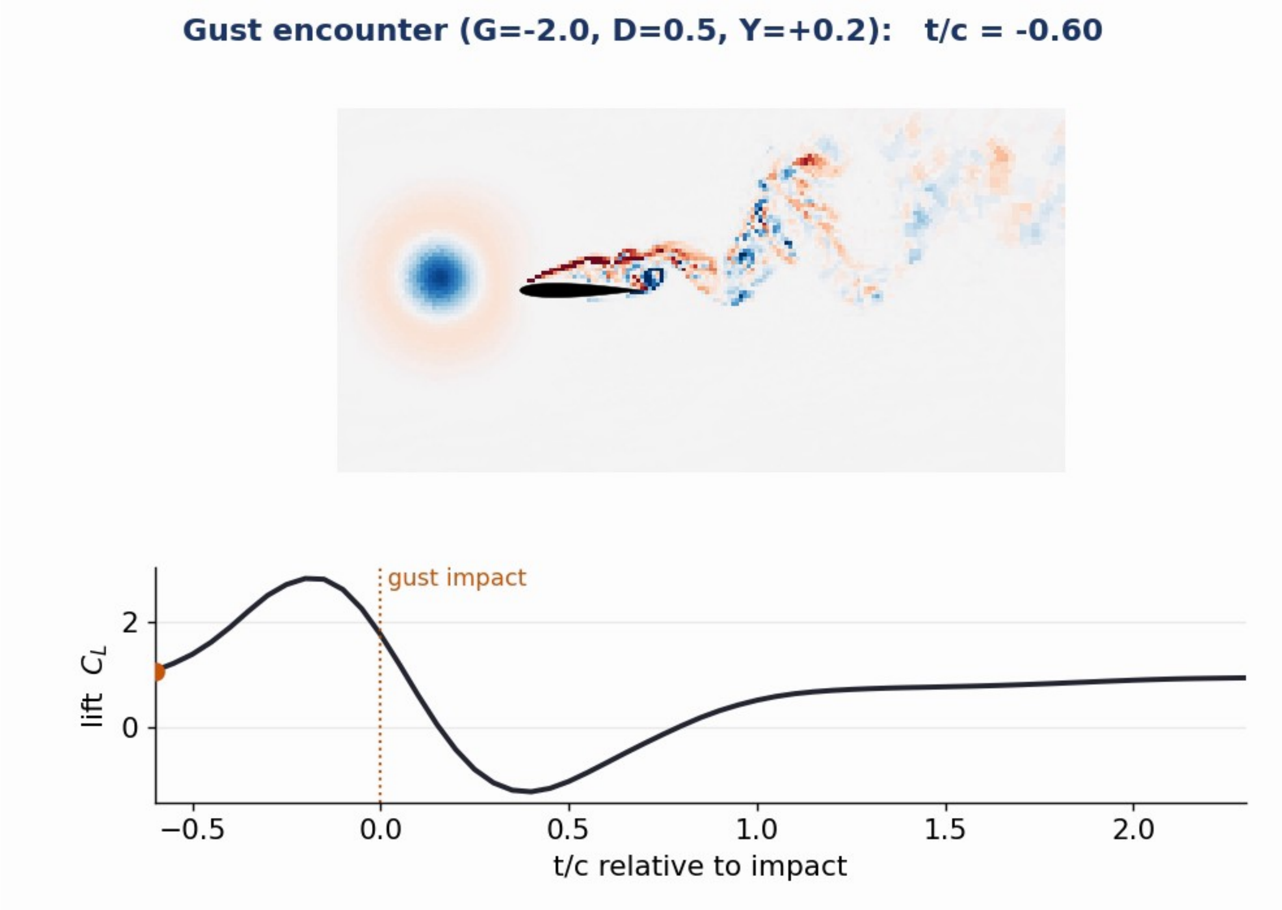
Sparse-sensing state estimation of vortex-gust airfoil interactions

Toward model-based flow control

A. Solera-Rico, A. Miró, O. Lehmkuhl, **C. Sanmiguel Vila**
csanvil@inta.es

The control problem: gusts are fast, strong, nonlinear

- Small and micro air vehicles increasingly fly where the **gust speed is comparable to or above the flight speed** (urban canyons, building and ship wakes): gust ratio **G > 1** brings large, fast load transients classical models miss
- A vortex strikes the wing: a **leading-edge vortex** forms and sheds, and the lift swings strongly (right: DNS, a moderate **G = -2** gust, C_L from about -1 to +3)
- Active control, model-based or RL, needs a **fast, faithful forward model** it can trust under rollout
- **This talk: what reduced state is worth planning against?**



Objective: a reduced state that keeps the wake

- The load transient is built by the **leading-edge vortex (LEV)** and the wake it leaves: the discriminating information is in the **wake**, not the integrated forces C_L, C_D
- We quantify it by the **large-scale wake enstrophy**: a Gaussian scale split (Motoori & Goto 2019) at $\sigma/c = 0.05$ isolates the load-bearing large-scale vorticity (LEV + shear layer) from fine turbulence, then integrates ω_L^2 over the wake
- **Objective:** find a reduced state that faithfully **encodes the wake** (the part reconstruction loses), not just the forces, and that is **recoverable from sparse sensors**
- And **compare candidate states**, predictive vs reconstructive vs linear, under one matched protocol

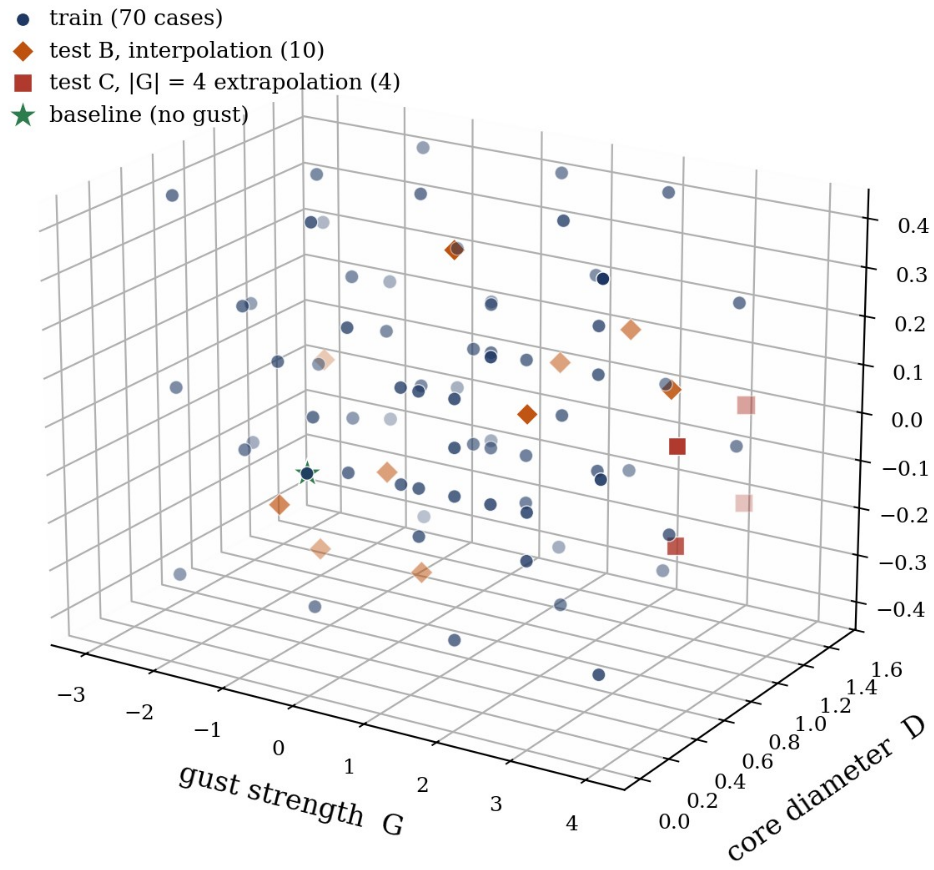
The opening for JEPA

- Most flow ROMs, POD/DMD, autoencoders, are trained to **reconstruct the field**
  - reconstruction fixes the latent only up to a diffeomorphism, so its geometry is not constrained to be predictable
- Alternative: learn a state that is **predictive by construction**, and plan against it

Which reduced state faithfully encodes the wake that reconstruction loses, and is recoverable from sparse sensors?

Data

- **DNS** (SOD2D, no subgrid-scale model), NACA 0012, α = 14°, **Re = 5000**
- Taylor-vortex gusts parametrised by strength **G**, core diameter **D**, wall-normal offset **Y** (right: the 84-case envelope, isometric)
- Impact-centred encounters; six observables: C_L, C_D, impulse I_y, **wake enstrophy**, ±circulation
- Held-out **test_b** (interpolation) and **test_c** (|G| = 4 extrapolation)



Training envelope in (G, D, Y): train, test_b interpolation, test_c |G| = 4 extrapolation, baseline.

What is a JEPA? (1 / 2)

Autoencoder (most flow ROMs): reconstruct the field

field $\omega(t)$ → Encoder → latent z → Decoder → reconstructed field

loss compares the reconstruction to the input in PIXEL space

JEPA: predict the next latent, no decoder

field $\omega(t)$ → Encoder → latent $z(t)$ → Predictor → predicted $z(t+1)$

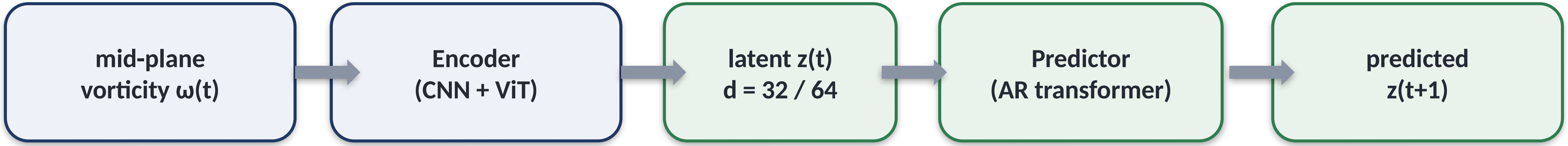
loss compares predicted $z(t+1)$ to $z(t+1)$ = Encoder(next frame) in LATENT space; an anti-collapse term replaces the decoder

Reconstruction ties the latent to pixels; prediction makes it forecastable, which is what a controller needs.

What is a JEPA? (2 / 2): the recipe and why for control

- **Training, in one line:** minimise the **latent** prediction error plus an **anti-collapse term** so the encoder cannot cheat by mapping everything to a constant (no decoder, no pixel loss)
- So the encoder **keeps only what predicts the future** (the control-relevant dynamics), and the predictor **gets no pixel-level shortcuts**
- **Cheap to roll out:** planning happens in the d = 32-64 latent, two to three orders of magnitude cheaper than evolving the full field
- Lineage: I-JEPA / V-JEPA (image, video); from-pixels LeWM, LeJEPA, PLDM, so far only gridworld and toy visual tasks
- **Why for control:** a learned world-model that is **observable from sensors**; to our knowledge the **first end-to-end JEPA on a parametric flow**

Our architecture for gust encounters



- The **encoder** maps the field to a latent: a pure state map $\omega(t) \rightarrow z$
- The **predictor** advances the latent $z(t) \rightarrow z(t+1)$ from the latent history; it **is** the latent dynamics model you plan or learn against
- A **visualisation decoder** is trained on the **frozen** encoder, never in the loss (figures only)
- Compared at matched latent dimension against a reconstructive AE (Fukami) and POD

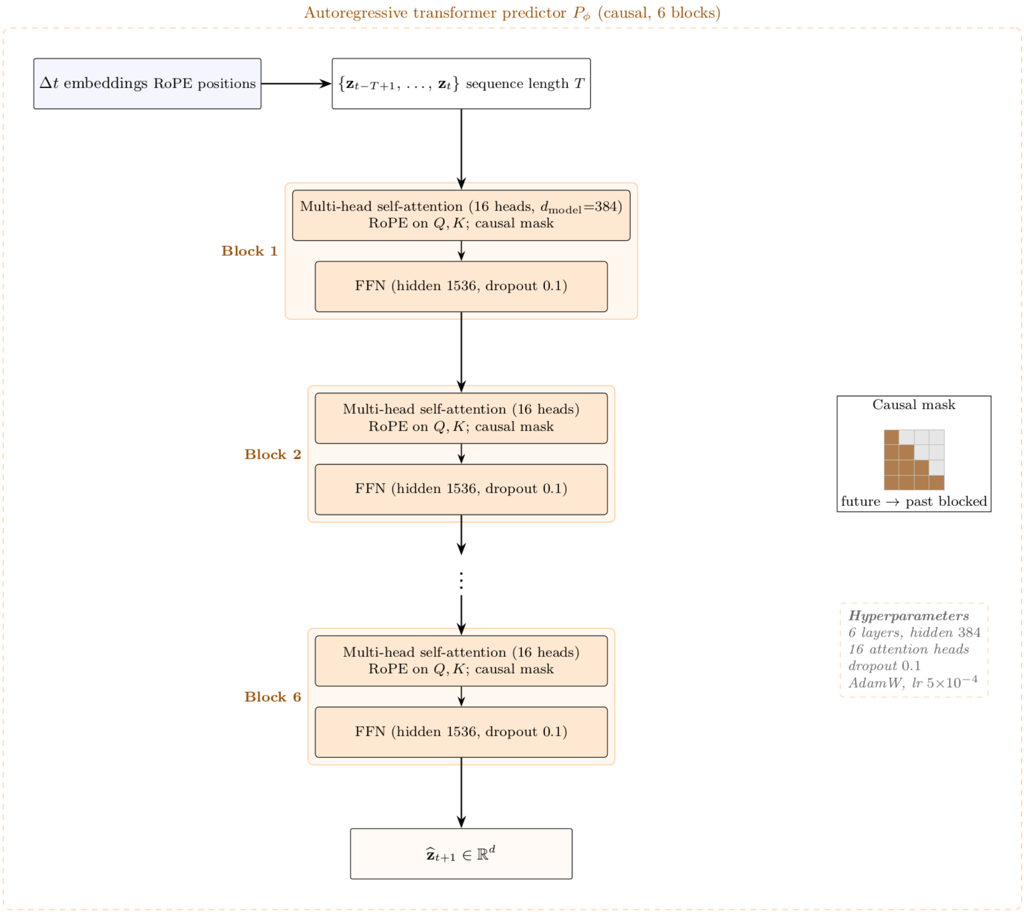
METHOD

The visualisation decoder (figures only, never in the loss)

- Trained on the **frozen** encoder, **never part of the JEPA loss**: it only renders a latent as a field so we can look at it
- Loss = a **region + Laplacian-pyramid + gradient + spectral-amplitude** objective (with small enstrophy / circulation terms), so it keeps the **large-scale wake structure** rather than just pixel MSE
- Held-out reconstruction **SSIM ≈ 0.73**

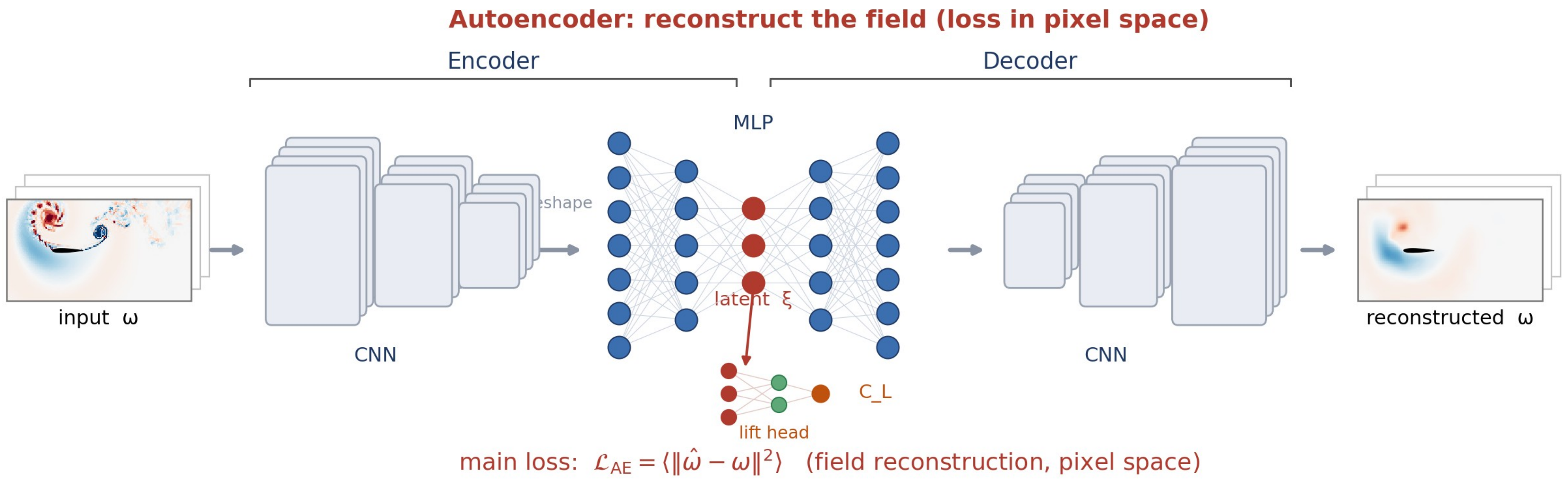
Model detail: encoder and predictor

- **Encoder** = a CNN stem (3 down-sampling stages) + a 6-layer Vision Transformer; the field becomes 288 patch tokens, and a learned [CLS] token is mapped by a small MLP (with BatchNorm) to the latent z (d = 32 / 64)
- **Predictor** = a 6-layer autoregressive transformer (hidden 384, 16 heads, dropout 0.1): it reads z(t) and emits the next latent z(t+1). **Unconditioned: no gust parameters enter the predictor**
- **Causal mask:** each step sees only past steps, never the future; **RoPE** encodes the frame ordering so the transformer knows what is earlier or later
- **Training:** match the next latent (teacher forcing) + **scheduled sampling** (feed the model its own predictions over an 8-step rollout, so it tolerates its own errors) + **SIGReg** anti-collapse (keeps z informative without a decoder); AdamW, bf16, 20k steps
- **Architecture inspired by the JEPA world-model line:** I-JEPA / V-JEPA, and the from-pixels LeWM (Maes et al. 2026) and LeJEPA (Balestriero and LeCun 2025)



Illustrated alternative: the autoencoder route

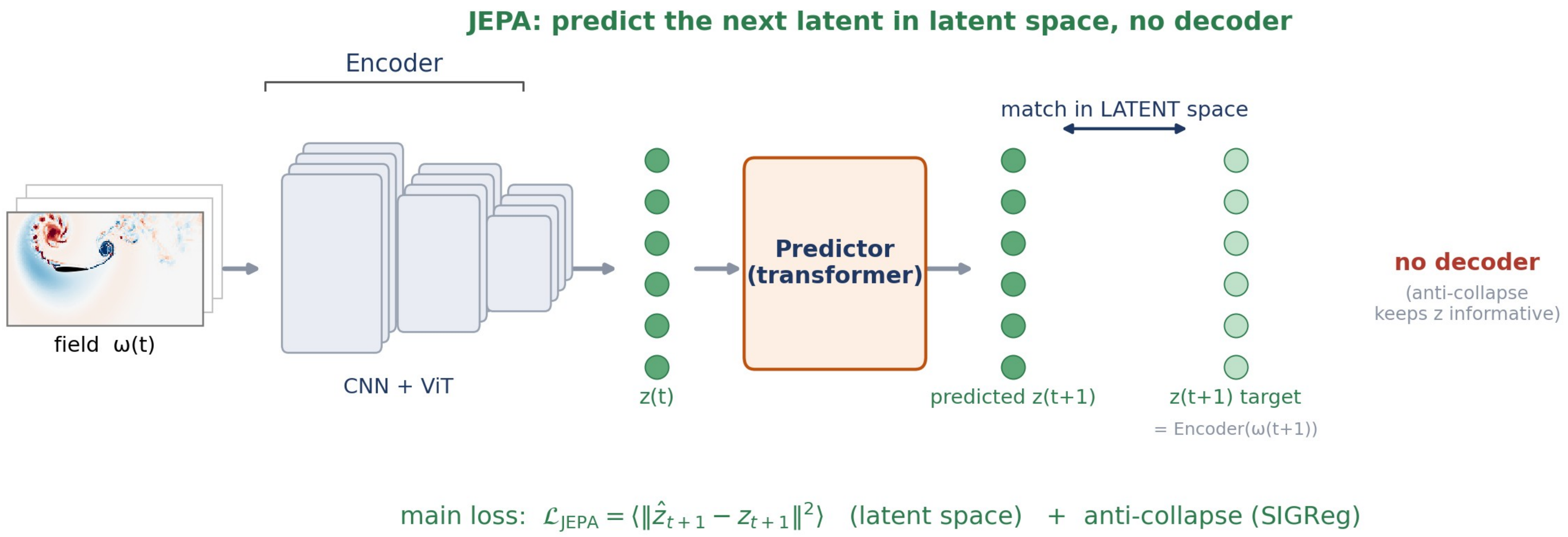
- Reconstructive route (Fukami-Taira lineage): a **CNN encoder**, an **MLP latent ξ** (which also feeds a lift head to C_L), and a **CNN decoder**; the loss compares the reconstructed field to the input in **pixel space**



An autoencoder reconstructs the field; the latent is shaped by pixel reconstruction.

Illustrated alternative: the JEPA route

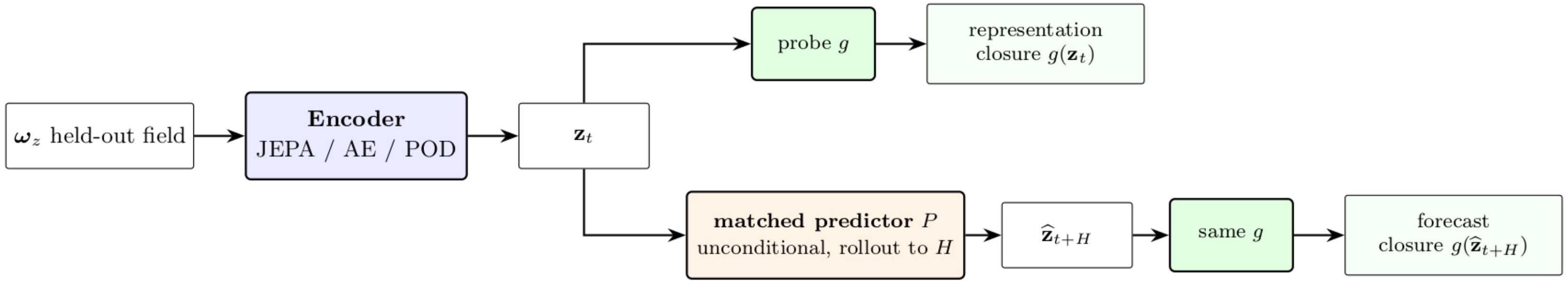
- Same encoder, but a **predictor** advances the latent and the loss lives in **latent space**, with **no decoder** (an anti-collapse term keeps z informative)



JEPA predicts the next latent in latent space, with no field reconstruction.

Closure protocol

- **Representational closure (the headline):** probe each observable from the **encoded** latent at impact + 16, what the state *carries*, no rollout
- **Forward (rollout) mode:** roll the predictor recursively to **H = 16** and probe the predicted latent; here we use it **qualitatively** (does the rolled latent track the wake?)
- **Matched protocol:** same predictor architecture and same probe family, **trained / fitted separately per latent**, so differences are the encoder's
- **Parameter-only floor:** can the **three gust numbers (G, D, Y) alone** predict each observable (a kernel-ridge fit)? Beating the floor proves the latent **carries flow state beyond the parameters**



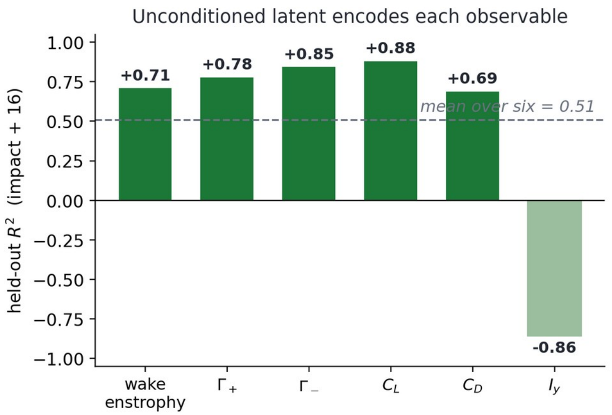
The predictor (unconditional, no gust parameters) and probe family g use the same architecture and protocol, trained separately for each encoder family.

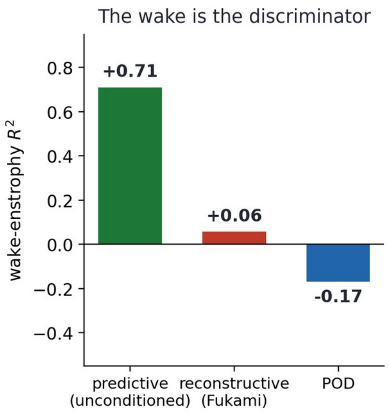
Same probe family maps every encoder (JEPA / AE / POD) to the observables. Representational mode (encoded latent) is the headline; forward rollout is shown qualitatively.

RESULT

Main result: the latent keeps the wake

- **Representational closure:** probe the **encoded** latent (no rollout) for each observable at impact + 16; the question is what the unconditioned state *carries*
- **Wake enstrophy is the discriminator: tf-no-c R^2 = 0.71** (the conditioned model 0.75; reconstructive and POD far lower, 0.06 and below)
- So the **unconditioned** latent (no gust parameters anywhere) **encodes the wake that reconstruction smooths away**
- Forces and circulations are read off cleanly too (C_L 0.88, $\Gamma_\pm$ 0.85 / 0.78, C_D 0.69); the **mean over six is 0.51**, with the transport impulse I_y the weak observable
- **Wake enstrophy** $E_w = \int \omega_z^2$ over the wake ($x/c \in [0.5, 4]$, $|y/c| \le 1$): the **intensity of the wake's rotational structures** (LEV + shed vortices) that set the future load, the part reconstruction-only states lose

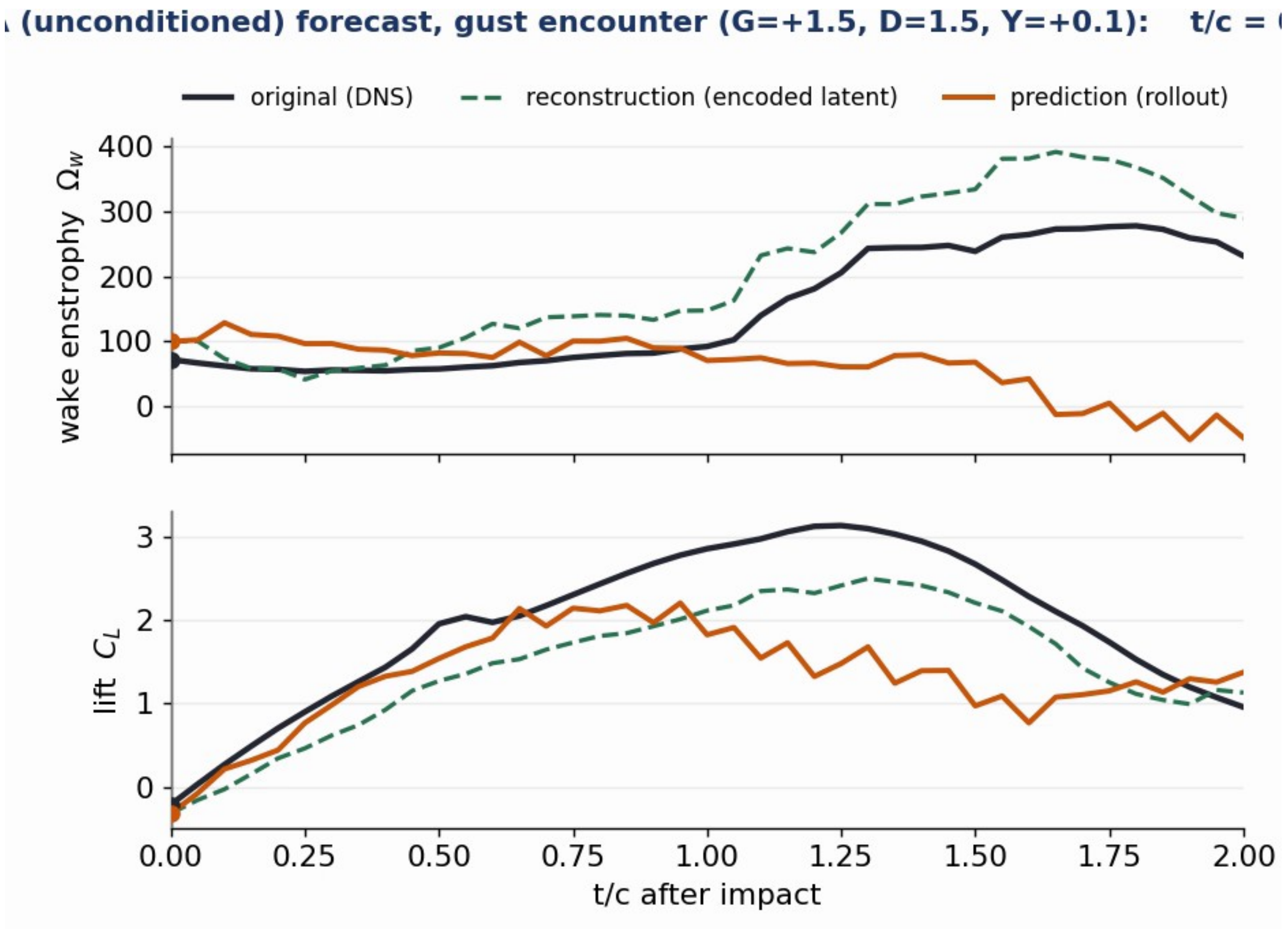




Held-out representational R^2 at impact + 16 from the encoded latent (test_b). Left: per observable (tf-no-c). Right: wake-enstrophy R^2 across families.

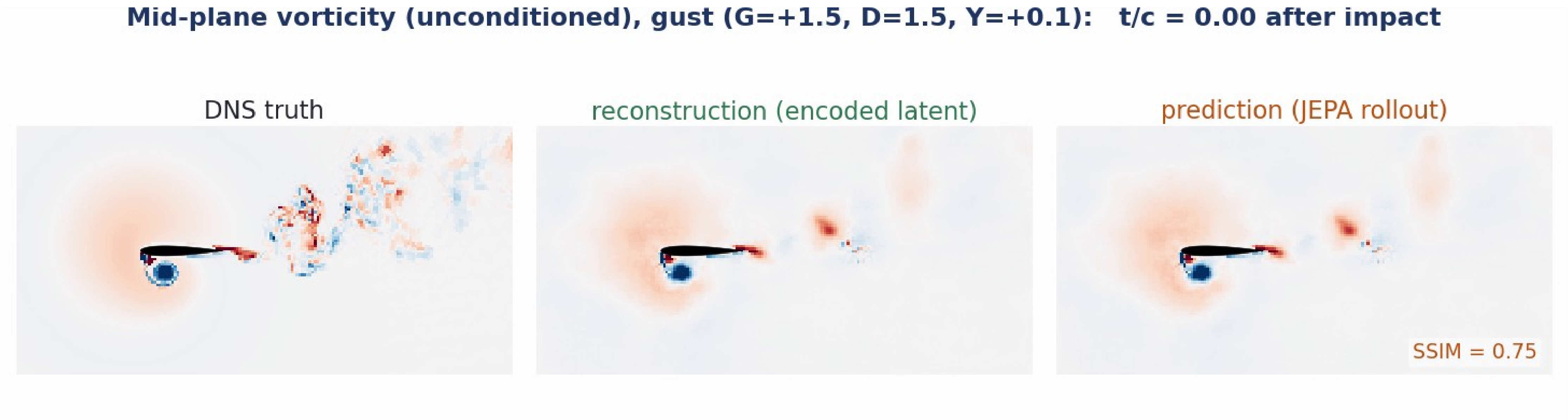
The rolled latent qualitatively tracks the wake

- The predictor **rolls the latent forward from impact**; each scalar is read off the rolled latent by a fixed linear probe
- **Wake enstrophy** and **lift** qualitatively track the DNS truth (black) through impact and the lift dip
- **Green = reconstruction** = the model's **own encoded latent decoded with NO rollout** (encode-then-decode of the true frame, the representational **ceiling**); it is **not** a separate reconstructive autoencoder
- **Orange = prediction** = the **rolled latent** (the predictor advanced forward, then decoded)
- A representative **low-error** encounter (G = +1.5), not the hardest case; shown qualitatively, no closure R^2 claimed here



The same rollout in physical space

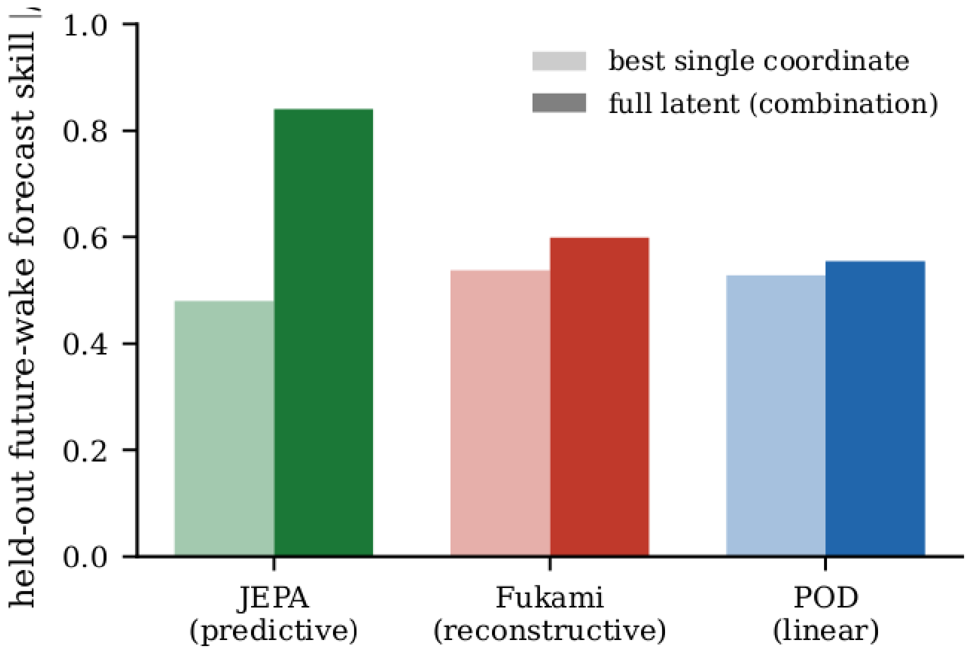
Decode the rolled latent every frame: the rolled latent qualitatively tracks the wake as full vorticity fields, not just scalars.



DNS truth vs reconstruction (JEPA encode→decode of the latent) vs prediction (rolled latent), G = +1.5. At impact (H = 0) prediction = reconstruction; they diverge with horizon. SSIM(prediction) 0.75 at impact, 0.63 at H = 16. The latent keeps the LEV and shear layer; fine-scale wake is not retained at d = 64.

It is the wake, not the forces

- Each bar is **how well the future wake is forecast**: **dark = the full latent** (all 64 coordinates together), **light = the single best coordinate**
- **Predictive JEPA:** the full latent (**0.84**) far exceeds the best single coordinate (**0.48**) → the wake is a **distributed code spread across many coordinates**
- **Reconstructive / POD:** the best single coordinate is **already as good as the whole latent** → there is **no distributed / collective wake code** to find
- **Forces** (C_L, C_D), by contrast, are carried **redundantly by every family**, in a single coordinate
- And it clears the **parameter-only floor** → the latent, not the parameters, does the work

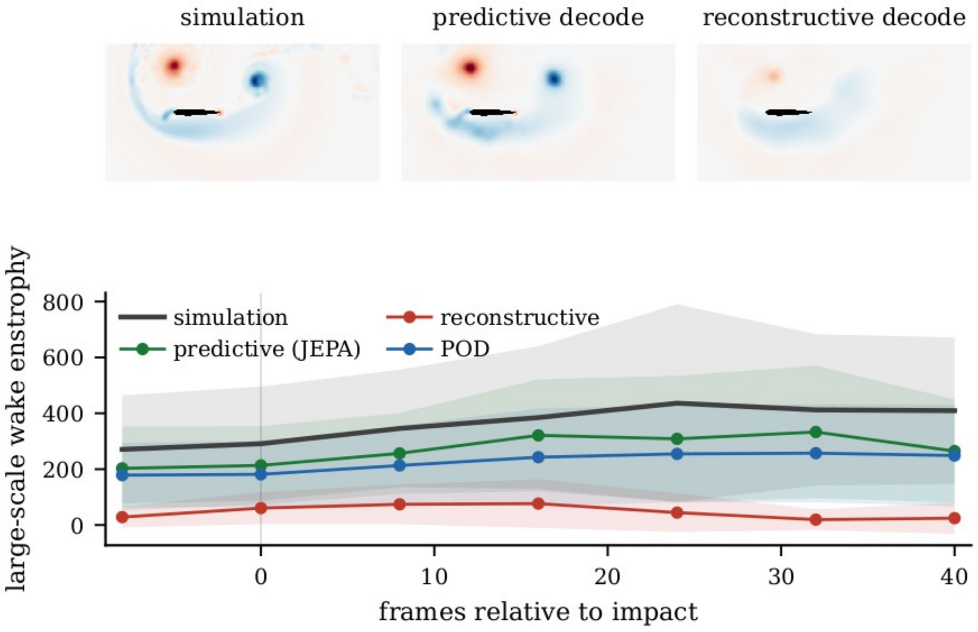


Future-wake skill: full latent (dark, all 64 coordinates) vs the single best coordinate (light). Predictive = distributed code; reconstructive / POD = one coordinate suffices.

RESULT

The wake in physical space: the Gaussian scale split

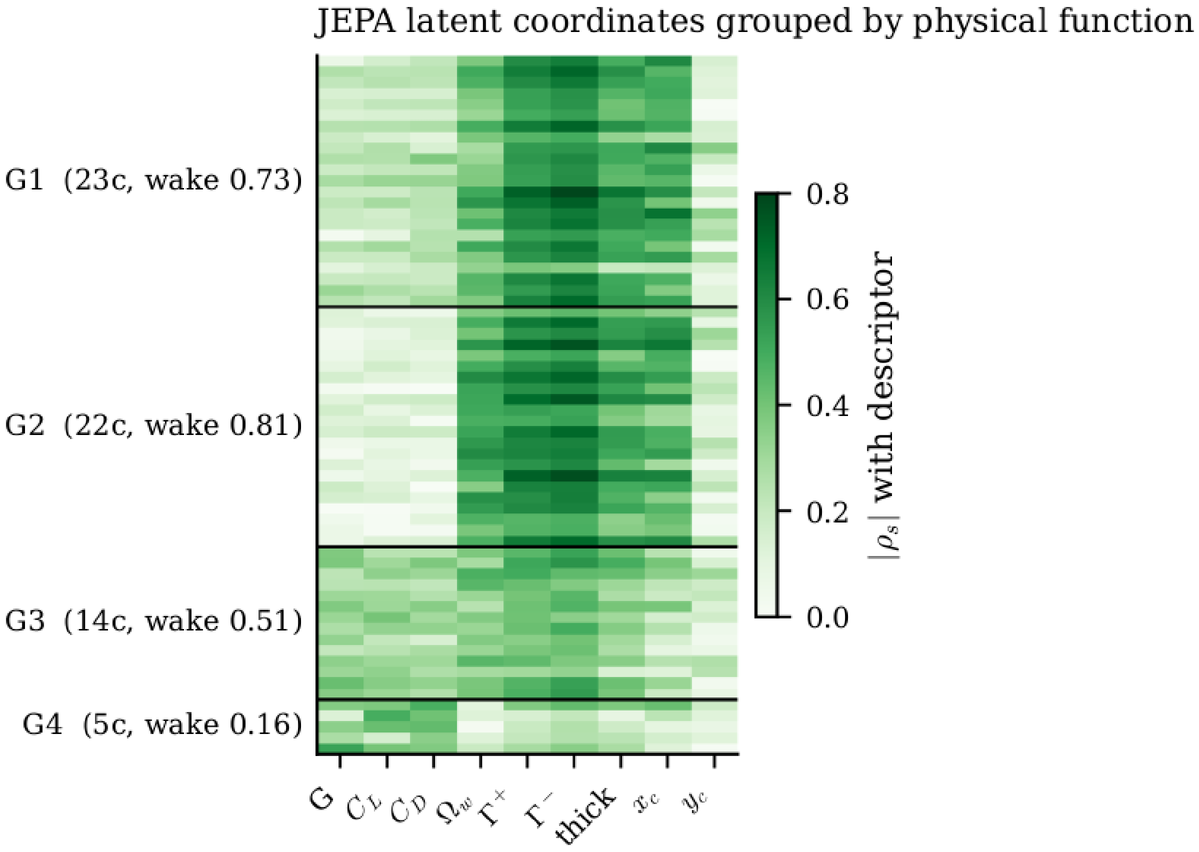
- **Gaussian scale split** (Motoori & Goto 2019): low-pass the vorticity at σ/c = 0.05 into a **large-scale** part (LEV + shear layer, carries the lift) and a fine part
- Top: large-scale vorticity at impact+16 for the strongest test gust (G = -3, D = 1.5, Y = -0.1), simulation vs the **predictive** and **reconstructive** encode-then-decode reconstructions: predictive keeps the LEV/wake, reconstructive smooths it
- Bottom: the **large-scale wake enstrophy** through the encounter; bands = mean ± 1 s.d. across test_b encounters
- So 'it is the wake' is visible in **physical space**, not only in the scalar closure



Large-scale (σ/c = 0.05) wake vorticity at impact+16 (top) and large-scale wake enstrophy vs frame (bottom), by family.

Latent coordinates group by physical function

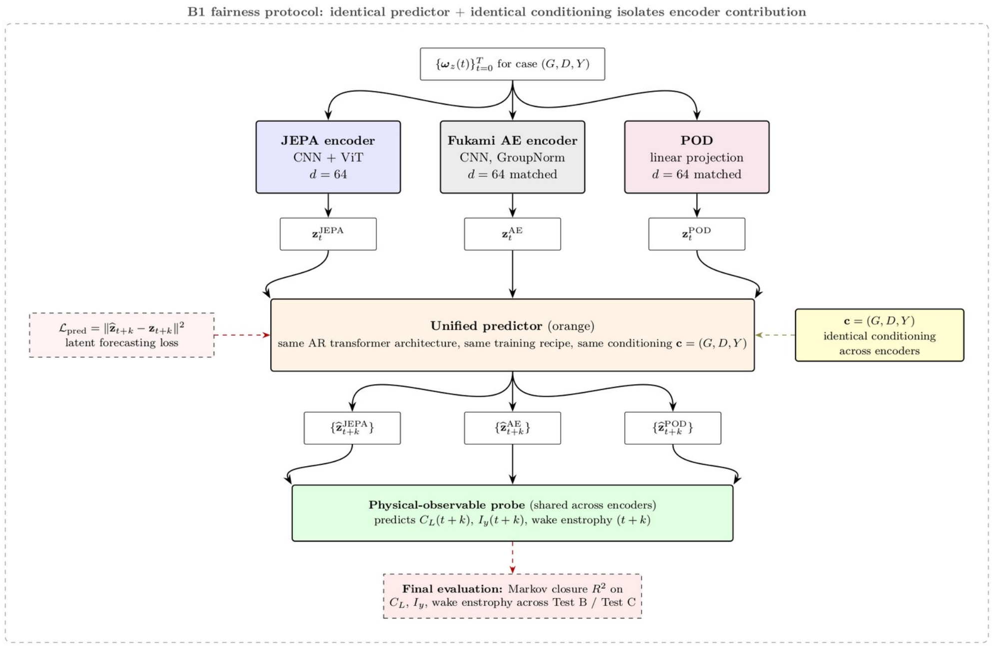
- Profile each of the 64 unconditioned-latent coordinates by its |Spearman| correlation with nine descriptors (gust G, forces, wake enstrophy, circulations Γ±, wake thickness, centroid)
- Clustering the profiles recovers **functional groups**: **~51 wake-vorticity** coordinates (three wake-dominated clusters) and **11 gust-forcing** coordinates (one force cluster)
- Alone, the wake groups carry the wake at **≈ 0.7-0.8**, the forcing group at **0.45** (all 64 together: 0.84), the collective wake code seen earlier
- Read **descriptively**, not as a causal claim



Per-coordinate |ρ| with each descriptor; rows grouped into functional clusters (G1-G4), labelled with coord count and held-out wake skill.

Controls: objective and supervision, not architecture

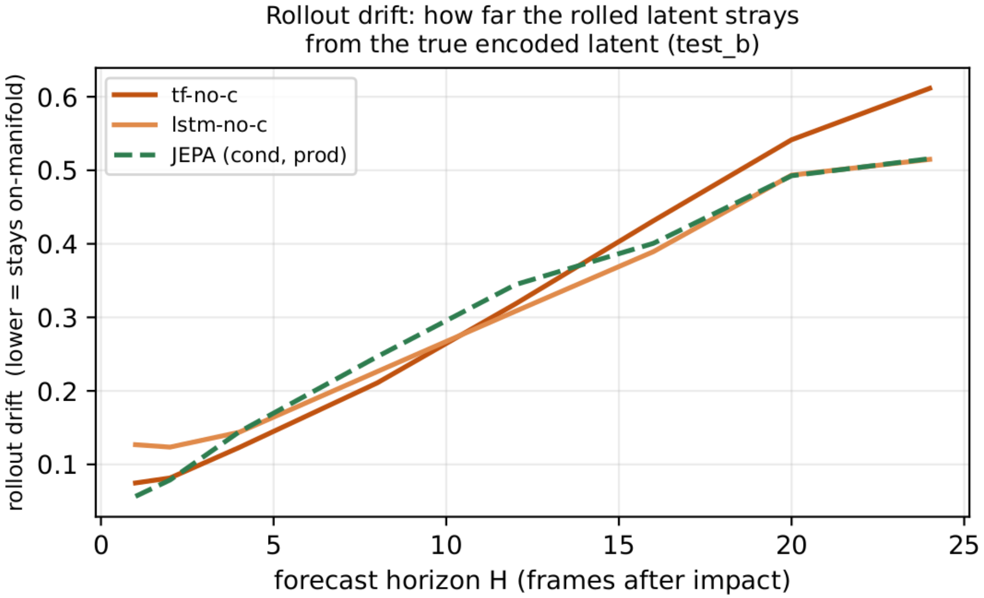
- **(Conditioned-model control; no unconditioned variant run.) 2 × 2:** objective {predictive, reconstructive} × architecture {CNN, CNN+ViT}, auxiliary heads matched
- **Fair comparison:** every cell is rolled with the **same temporal predictor** (the matched closure predictor: same architecture and protocol, trained separately per family), so the **only thing that varies is the encoder's objective / architecture**
- The **predictive objective wins at both architectures** (wake R^2 0.46 / 0.45 vs 0.16 / 0.29) → **it is the objective, not the ViT**
- And the **wake-observable head is a necessary ingredient**: removing it removes the gain
- → the gain needs the **predictive objective and wake supervision together**



Objective × architecture controls, three seeds per cell; every cell rolled with the same matched closure predictor, so only the encoder's objective / architecture varies.

Diagnostic 1: latent drift

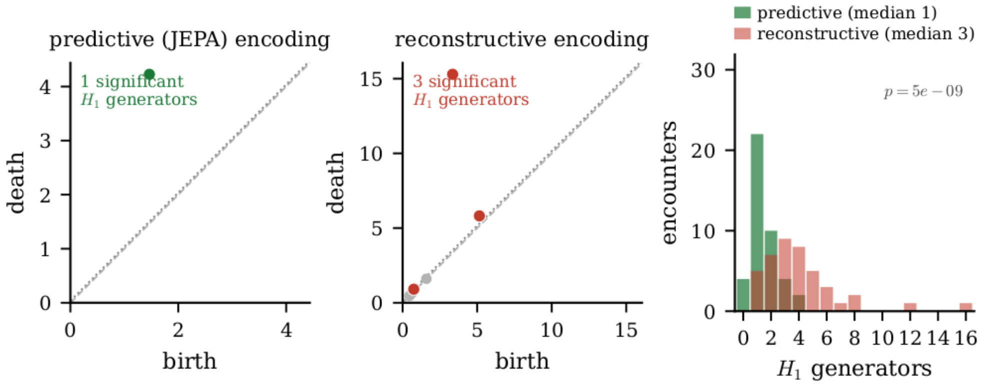
- **Why this question:** a planner / RL agent queries the model at states reached by its **own rollout**, one-step error is not enough
- **z_dns** = the latent **encoded directly from the simulation field** (ground truth); **z_markov** = the latent **produced by rolling the predictor forward**
- The curve is their **relative distance vs horizon** (‖z_markov − z_dns‖ / ‖z_dns‖); **lower / flatter is better** (the rollout stays on the data manifold)
- **Result:** the **unconditioned** rollout drift grows **gracefully** and **matches the conditioned model**; it stays on-manifold, exactly where the probes (and a planner) are valid



Rollout drift vs horizon (test_b): how far the rolled latent z_markov strays from the true encoded latent z_dns (relative distance). Lower / flatter = stays on-manifold.

Diagnostic 2: topology of the encounter

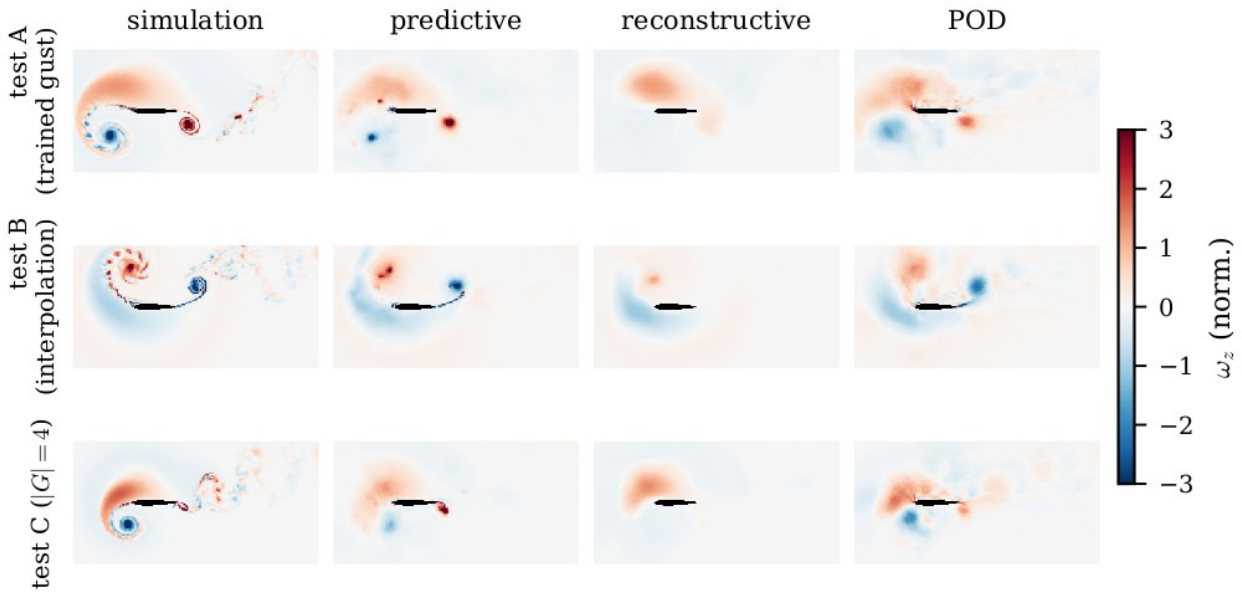
- A gust encounter is a **cycle**: the flow loops and roughly returns to where it started
- We ask whether the latent traces **one clean loop** or a **tangle of many**
- **Persistent homology** is just a principled **loop-counter** on the latent path (it counts the loops that genuinely persist, not noise)
- **Result:** the **unconditioned** JEPA latent makes **one loop** (median 1); the **reconstructive** latent **fragments into many** (median ~4); p ≈ 5×10^{-9}
- So the predictive state captures the encounter as a **single coherent cycle** (count of persistent H1 generators; Mann-Whitney test)



The latent path of an encounter: the predictive latent traces one loop (median 1), the reconstructive latent many (median ~4). Persistent homology counts the loops.

Decoded reconstructions

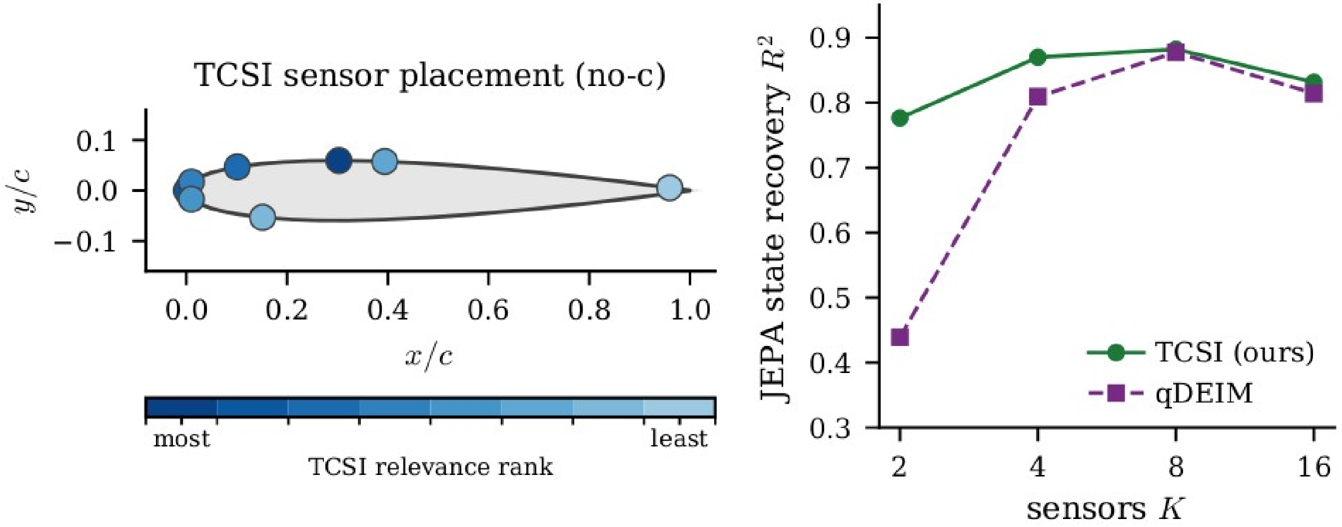
- Decoded fields: predictive vs reconstructive vs POD vs DNS
- The predictive decode is **blurrier pixel-wise** but preserves the **transported large-scale wake**; the reconstructive decode is sharp yet drifts under rollout



Held-out reconstructions; the predictive objective trades pixel sharpness for transported structure.

Sparse sensor placement

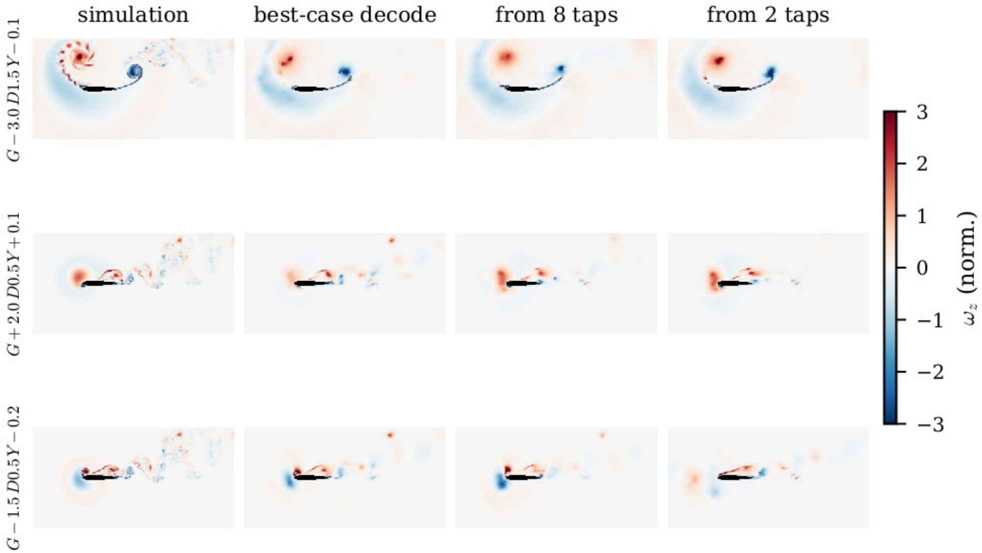
- Sparse wall-pressure sensors selected by **TCSI / qDEIM** vs uniform
- **Model:** the pressure → latent map is a **KernelRidge (RBF)** regressor on the K-sensor × pre-impact-window vector; K = 2 / 4 / 8 / 16 (LSTM / KRR 5-fold-CV selected to guard small-sample overfitting)
- **Metric — state-recovery R^2:** the **fraction of the held-out latent's variance recovered from the K pressure taps** (1 = perfect, 0 = no better than the mean), for the **unconditioned tf-no-c latent** (no gust parameters anywhere)



Optimal sparse sensor placement on the airfoil surface; state-recovery R^2 is the fraction of the unconditioned latent's variance recovered from K taps.

Flow recovered from sparse wall pressure

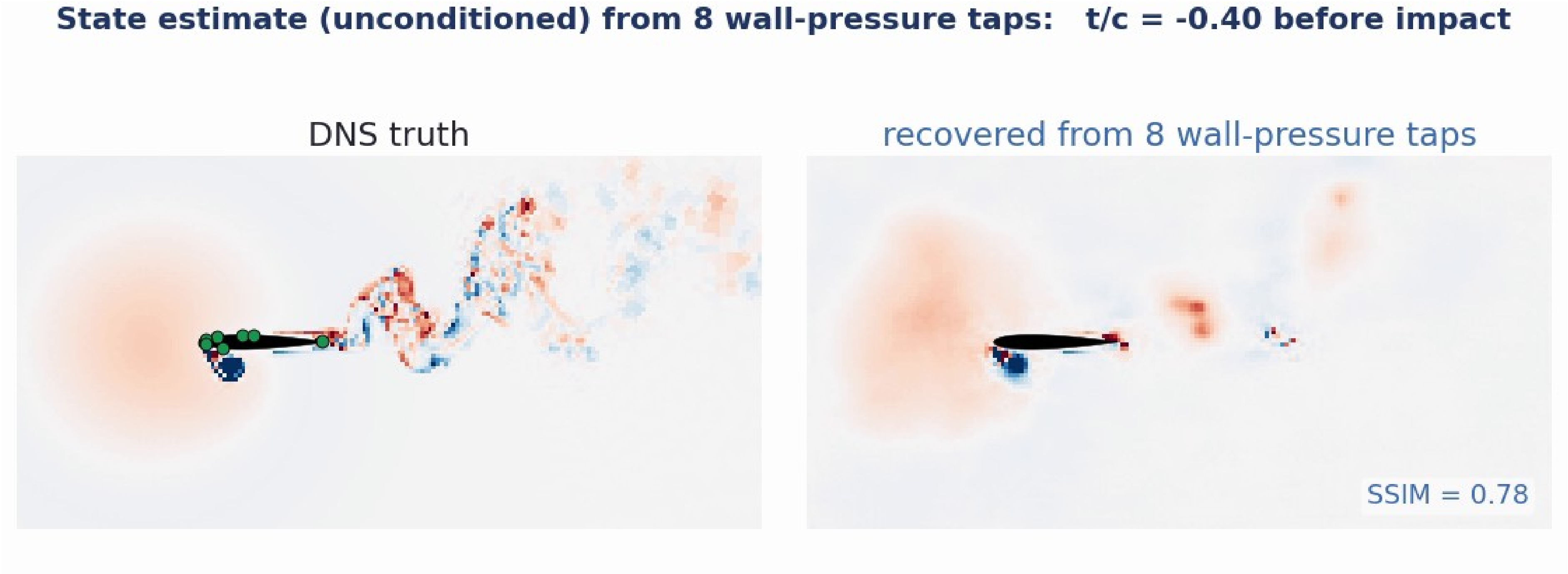
- **Model:** a **KernelRidge (RBF kernel)** regressor maps the K wall-pressure taps over a pre-impact window to the **impact-frame latent**; the frozen decoder then renders the field
- **K = 8 taps** recover the leading-edge vortex and shear layer; **K = 2** coarsens but keeps the gross wake structure
- Benchmarked against the **best-case decode** (decode of the **true simulation latent**, i.e. the ceiling decode) and the simulation
- So the predictive state is **reconstructible from a few wall sensors**, a deployment-relevant observability result



Flow recovered from sparse wall pressure: simulation, best-case (ceiling) decode of the true simulation latent, and decode of the latent estimated from K = 8 and K = 2 taps (held-out).

Sparse-sensor state estimation in action

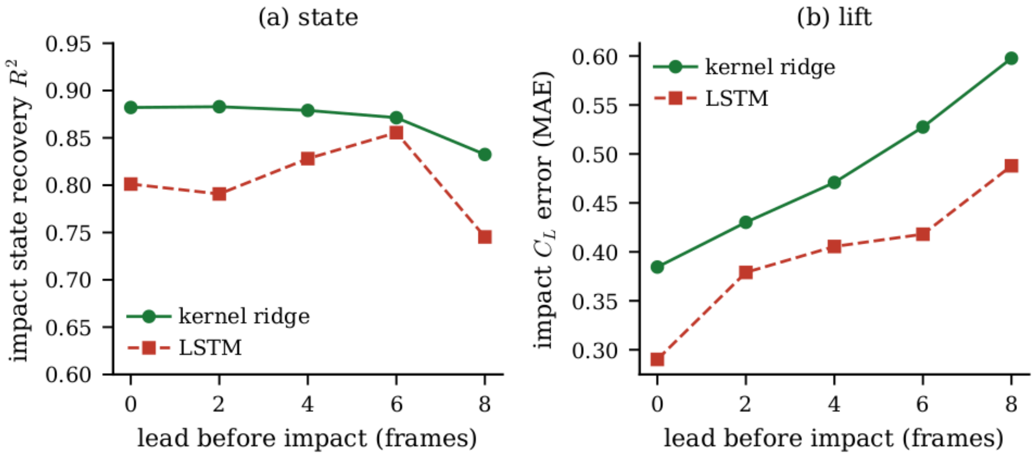
No predictor here: a per-frame **MLP** maps a causal 6-frame window of 8 wall-pressure taps to the latent, then the frozen decoder renders the field.



DNS truth (green dots mark the 8 TCSI taps) vs the field recovered from wall pressure alone, on a held-out encounter. Held-out-encounter latent R^2 = 0.74, SSIM 0.63. Wall pressure fixes the near-body LEV and shear layer; the far wake is not surface-observable.

Control relevance: observable ahead of impact

- From a causal window of sparse wall pressure, recover the **impact-frame latent** at a **lead time before impact** (held-out test_b)
- **Impact state** is recovered at **R^2 ≈ 0.88** right at impact and stays **above 0.83 out to 8 frames ahead** (kernel ridge; LSTM comparable)
- **Impact lift** C_L is estimated with MAE rising gracefully from **≈ 0.38 at impact to ≈ 0.60 at 8 frames ahead** (LSTM lower, 0.29 to 0.49)
- So the **unconditioned** state is **observable ahead of impact** from a few wall sensors: the ingredient a controller consumes

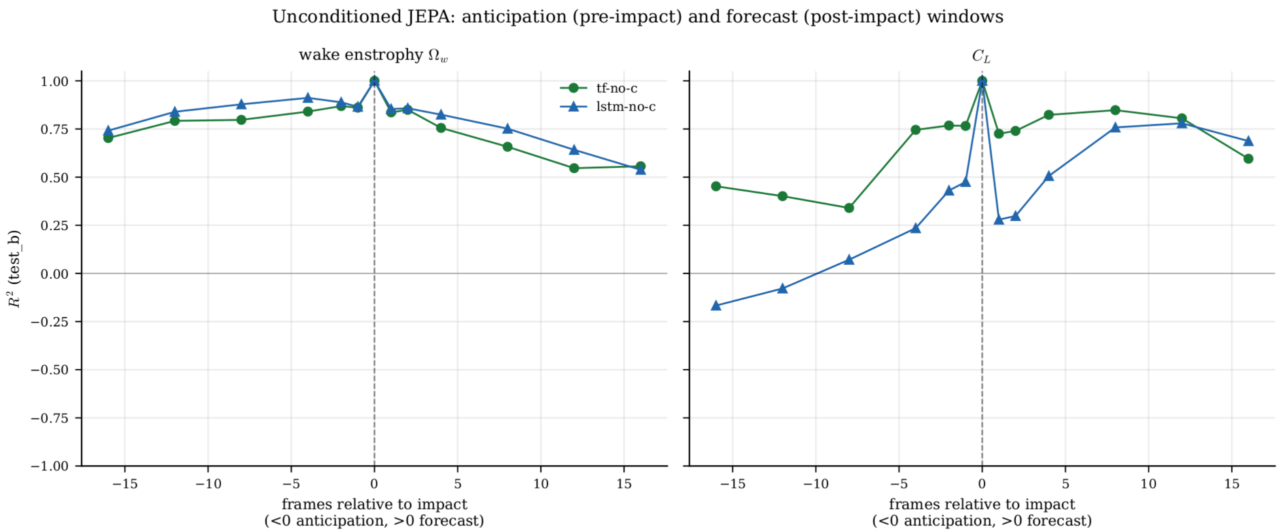


Recovered from sparse wall pressure at each lead time before impact (test_b): (a) impact-state R^2; (b) impact-C_L MAE. Kernel ridge vs LSTM.

CONTROL RELEVANCE

Two forecast windows: early warning, then forecast

- We **roll the unconditioned JEPA's own predictor forward in latent space** and read each observable off the rolled latent with a **fixed linear probe**: we **predict the JEPA latent, then probe wake / lift from it**
- **Wake:** a wide **±16-frame (≈ ±0.8 t/c)** window around impact (R^2 0.7-0.9) -- the structural signature gives **early warning** and short-range forecast
- **Lift C_L:** hard to anticipate (flat before impact) but **strongly forecastable after** (R^2 ≈ 0.8 to +12) -- the load is predictable **once impact hits**



Wake (left) and lift (right) R^2 vs frames relative to impact (test_b): x < 0 anticipation lead, x > 0 forecast horizon; tf-no-c and lstm-no-c.

Conclusions

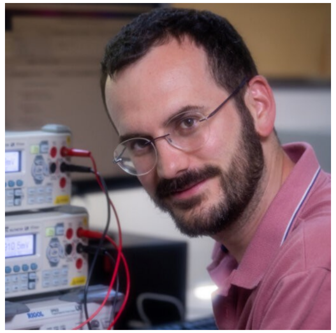
- A **fully unconditioned predictive latent** (no gust parameters anywhere) **representationally keeps the wake** (wake $R^2 \approx 0.71$) that **force-only and reconstruction-only** states lose
- That latent **traces a single clean cycle**, is **transport-consistent** within an encounter, and is **observable from sparse wall pressure**
  - **compact** ($d = 32 \approx d = 64$, participation ratio ≈ 1.7)
- The **rollout** is shown **qualitatively**: it stays on-manifold and tracks the wake, but we make **no forward-closure R^2 claim**
- It is a **substrate** for model-based control and RL world-models, **not yet a validated controller**
- **Next:** add an **actuation channel** (same machinery), **close the loop**, push to **3D observability** at $|G| = 4$, and carry the recipe to **other parametric flows**

Acknowledgements

This work is a collaboration with:

Alberto Solera-Rico

INTA · UC3M



Arnau Miró

UPC · BSC

Oriol Lehmkuhl

BSC

Work produced with the support of a 2024 Leonardo Grant for Researchers and Cultural Creators, BBVA Foundation, project PREVENT (grant LEO24-2-15988). The Foundation takes no responsibility for the opinions, statements and contents of this project, which are entirely the responsibility of its authors.

Fundación BBVA

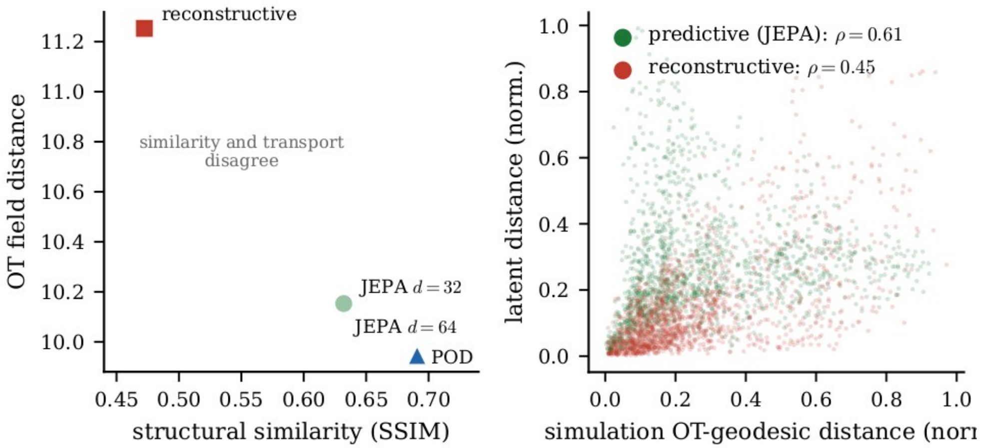
RED LEONARDO
BECAS DE INVESTIGACIÓN CIENTÍFICA Y CREACIÓN CULTURAL

Backup slides

Supporting detail and ablations

Transport geometry (optimal transport)

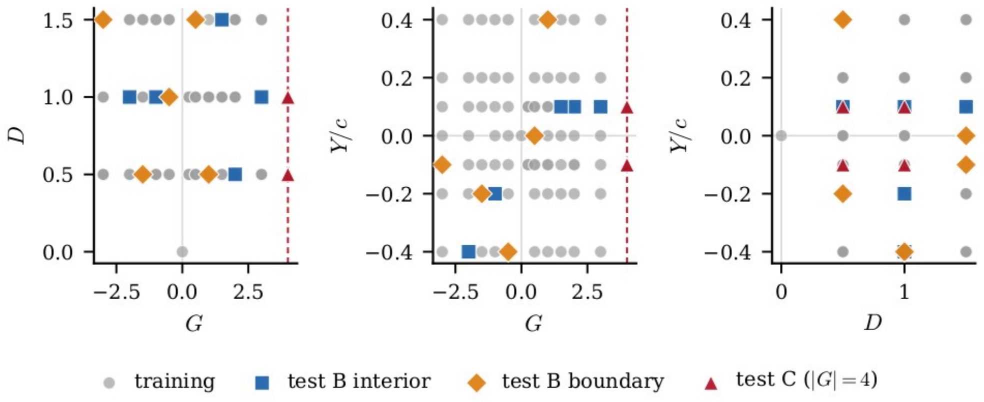
- **Why:** Euclidean / pixel distance ignores **where structures move**; control cares about advection of the LEV and shear layer
- **OT field distance** (after Tran, Yeh & Taira, JFM 2026): split vorticity into ± parts, transport each with **unbalanced Sinkhorn** OT and sum, the least work to rearrange one field into another
- We did **not** train the latent to match OT (that paper does); this is a **post-hoc test**: per encounter, Spearman-correlate the **latent** distance matrix with the **OT** matrix (n = 42)
- **Within-encounter:** the unconditioned JEPA **0.61** vs reconstructive **0.45**, order-preservation along the trajectory, **not** an isometry
- **Honest caveat:** *pooled across* encounters the alignment **does not hold**, so it is **trajectory-local**, not a global latent metric



Per-encounter latent-vs-OT distance alignment (Shepard). OT field distance after Tran, Yeh & Taira (JFM 2026). Within-encounter Spearman 0.61 vs 0.45; pooled it does not hold.

Dataset and protocol

- Partition **v2** (locked, sha256-anchored): 84 cases
- **226** train encounters · **42** test_b · **24** test_c (|G| = 4)
- ω_z pipeline v1: spatial mask + p99.99 clip + 3σ scale (train_std 3.55), no mean shift (preserves vorticity antisymmetry)
- Reporting: bootstrap n = 2000 CIs · 3 encoder seeds · 5-fold probe CV



Training envelope in (G, D, Y).

Matched-capacity: d = 32 vs d = 64

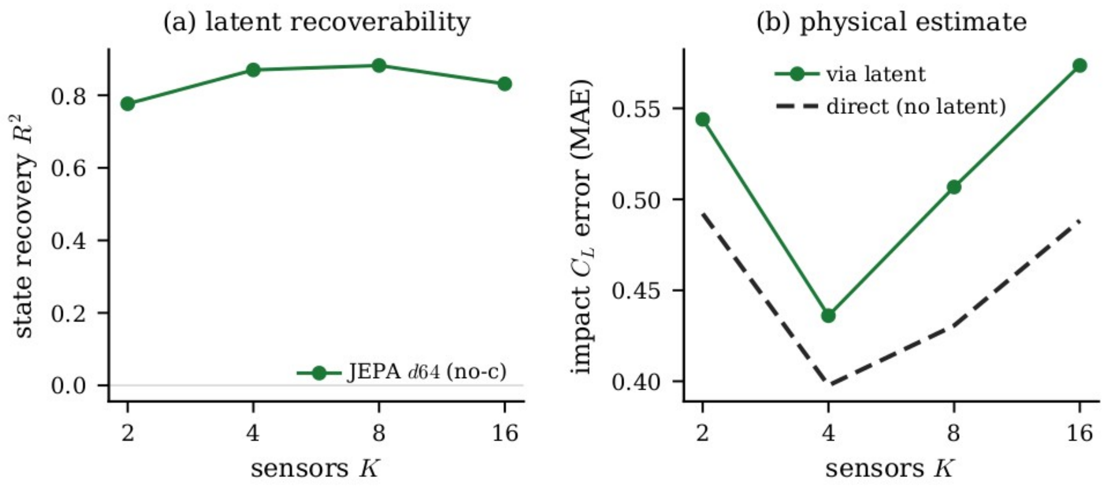
- **(Conditioned-model control; no unconditioned variant run.)** Halving the latent leaves the representation and **every mechanism diagnostic** intact
  - representational wake R^2 0.74 (d=32) vs 0.75 (d=64); drift ratio 0.86 vs 0.85; OT 0.61 vs 0.63
- Cost is **in-distribution forecast sharpness**: H = 16 wake closure 0.45 → 0.21
- Participation ratio ≈ 1.7, leading PC ≈ ¾ of variance → the effective dimension is a handful

Seed variance

- **(Conditioned-model control; no unconditioned variant run.)** Three encoder seeds per cell; wake-closure means reported ± standard deviation
- The reconstructive CNN+ViT cell has **large seed variance (±0.27)**, consistent with the drift mechanism: without a forward-predictable geometry, closure is unstable across initialisations
- Predictive cells are tight (±0.03–0.06)

Parameter observability from the latent

- z → (G, D, Y) probe R^2 from the rolled-out latent, K = 8 pre-impact sensors
- test_b: **G 0.46 · D 0.80 · Y 0.10**, gust impulse and depth recoverable; cross-stream offset Y is marginal
- test_c (|G| = 4): negative, the 3D observability boundary of a single mid-plane slice



Gust parameters implicit in the latent on held-out cases.

SSIM convention

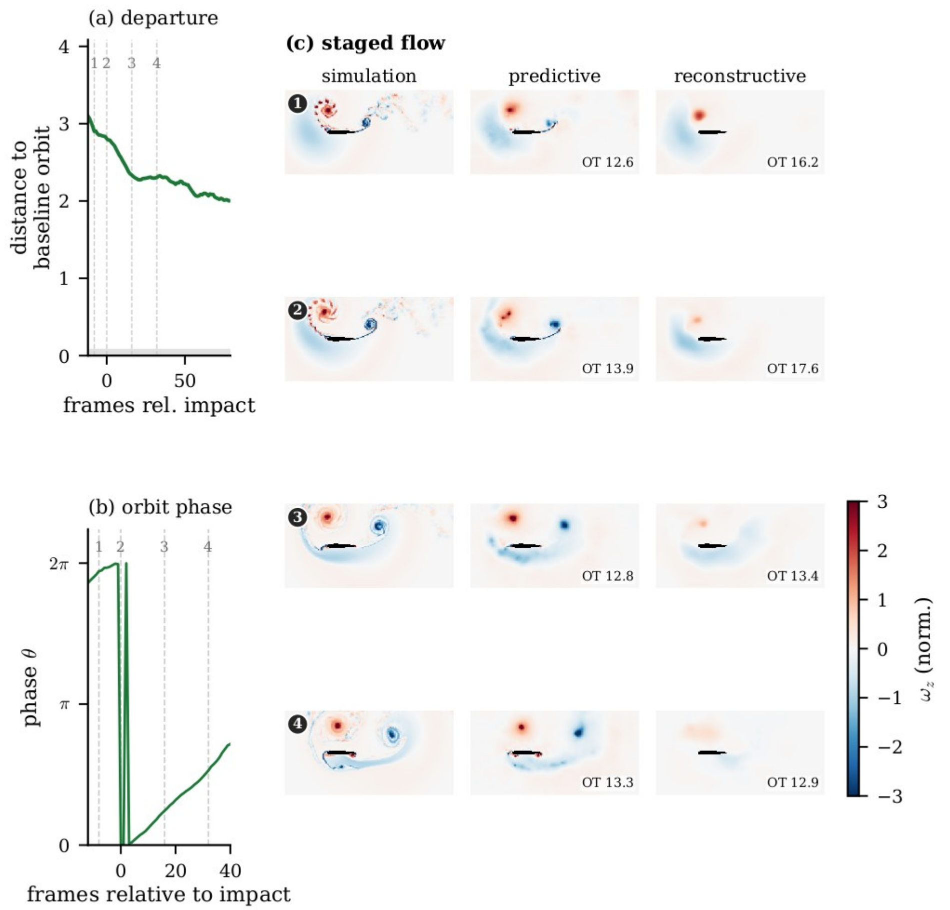
- Reconstruction SSIM uses the **Wang convention** ($K1 = 0.01$, $K2 = 0.03$) on pipeline-normalised ω
- Data range **$L = 2 \cdot \text{global p99.9}(|\text{target}|) \approx 8.31$** (split v2)
- Decoder test_a SSIM ≈ **0.73** at this convention

Parameter-only floor

- KRR-RBF on (G, D, Y) → observable at impact; train / test_b / test_c R^2
  - on train, (G, D, Y) interpolates C_L and C_D well (226 points in 3-D)
- On **test_b** the floor **collapses** across observables; on **test_c** it is negative for 5 of 6
- → JEPA's generalisation is **not** explained by the gust parameters alone

Phase–amplitude reading

- Phase–amplitude decomposition of the encounter cycle in the predictive latent
- The sharp **2π → 0 step** in panel (b) is a **phase wrap** (θ is a cyclic angle on the orbit; it resets at the branch cut, which falls near impact), **not a physical jump**
- Connects the predictor to the **sensitivity-function control** designed for these flows
- An actuation channel is the natural next input for closed-loop control



The 2π → 0 step in panel (b) is a phase wrap (cyclic angle resetting at the branch cut near impact), not a physical discontinuity.